

Método Robusto para Download de Arquivos em Aplicações Flet Web

O método mais robusto e recomendado para gerar arquivos no backend e oferecer downloads em aplicações Flet web (localhost) que funciona consistentemente tanto no ambiente de desenvolvimento (`flet run`) quanto em aplicações empacotadas com PyInstaller (`--noconsole`) é a combinação de **FastAPI endpoints** com **`page.launch_url()`**.

Arquitetura Recomendada

1. Estrutura do Backend com FastAPI

A implementação ideal utiliza o padrão de integração Flet + FastAPI para criar endpoints dedicados ao download de arquivos^{[1][2]}:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from contextlib import asynccontextmanager
import flet as ft
import flet_fastapi
import os
import tempfile

@asynccontextmanager
async def lifespan(app: FastAPI):
    await flet_fastapi.app_manager.start()
    yield
    await flet_fastapi.app_manager.shutdown()

app = FastAPI(lifespan=lifespan)

@app.get('/download/{filename}')
async def download_file(filename: str):
    # Lógica para gerar ou localizar o arquivo
    file_path = prepare_file(filename)

    headers = {
        'Content-Disposition': f'attachment; filename="{filename}"'
    }
    return FileResponse(
        path=file_path,
        headers=headers,
        media_type='application/octet-stream'
    )

# Montar a aplicação Flet
app.mount('/', flet_fastapi.app(main_app))
```

2. Configuração de Ambiente

Para garantir funcionalidade consistente entre desenvolvimento e produção, é essencial configurar adequadamente as variáveis de ambiente^{[3][4]}:

```
import os

# Configurar chave secreta para uploads (obrigatório)
def set_environment(variable, default):
    value = os.getenv(variable, default=default)
    os.environ[variable] = default if value is None else value

set_environment("FLET_SECRET_KEY", os.urandom(12).hex())
```

3. Gerenciamento de Arquivos Temporários

Para aplicações robustas, utilize os diretórios apropriados fornecidos pelo Flet^[5]:

```
import os
import tempfile

# Usar diretórios do Flet para persistência
app_data_path = os.getenv("FLET_APP_STORAGE_DATA")
app_temp_path = os.getenv("FLET_APP_STORAGE_TEMP")

def prepare_file(filename):
    # Para arquivos temporários com nomes customizados
    with tempfile.TemporaryDirectory() as temp_dir:
        file_path = os.path.join(temp_dir, filename)
        # Lógica de geração do arquivo
        return file_path
```

Implementação do Frontend

Método Recomendado: `page.launch_url()`

O controle ideal para trigger do download é o `page.launch_url()`, que funciona consistentemente em todos os ambientes^{[1][2]}:

```
async def main(page: ft.Page):
    async def download_file(e):
        # Para aplicações async (FastAPI), usar launch_url_async
        await page.launch_url_async(
            url='/download/meu_arquivo.xlsx',
            web_window_name='_self'
        )

    download_button = ft.ElevatedButton(
        "Download Arquivo",
        on_click=download_file
    )

    await page.add_async(download_button)
```

Configuração do FileResponse

Para garantir que o navegador exiba o diálogo "Save As", é crucial configurar adequadamente os headers HTTP^[2]:

```
headers = {
    'Content-Disposition': 'attachment; filename="nome_arquivo.ext"'
}
return FileResponse(
    path=file_path,
    headers=headers,
    media_type='application/octet-stream', # Força download
    filename=filename
)
```

Problemas Comuns e Soluções

1. App Restarts

Problema: Aplicação reinicia durante o processo de download.

Solução: Utilizar gerenciamento adequado de contexto com `asynccontextmanager` e garantir que o endpoint não interfira no ciclo de vida da aplicação Flet^[6].

2. Route Not Found Errors

Problema: Erros 404 ao acessar endpoints de download.

Solução:

- Verificar se o endpoint FastAPI está montado corretamente antes da aplicação Flet
- Usar paths absolutos nos endpoints (`/download/` não `/download`)
- Configurar adequadamente o `upload_endpoint_path` se necessário^[7]

3. Browser Blocking

Problema: Navegador bloqueia downloads por questões de segurança.

Soluções:

- Configurar adequadamente os headers `Content-Disposition`
- Usar `media_type='application/octet-stream'` para forçar download
- Para desenvolvimento local, alguns navegadores podem requerer HTTPS^{[8][9]}
- Testar com diferentes navegadores para identificar políticas específicas

Diferenças: Desenvolvimento vs PyInstaller

Ambiente de Desenvolvimento (flet run)

- Paths relativos funcionam normalmente
- Acesso direto ao sistema de arquivos
- Hot reload disponível^[10]

Aplicação Empacotada (--noconsole)

- Usar paths absolutos ou relativos ao executável
- Considerar que recursos podem estar embedados^{[11][12]}
- Testar especificamente o comportamento de arquivos temporários
- Usar --noconsole ou --windowed adequadamente para evitar janelas de console^[13]

Configuração de Produção

```
# Configuração robusta para produção
def create_app():
    # Configurar upload directory se necessário
    upload_dir = os.path.join(os.getcwd(), "uploads")

    app = flet_fastapi.FastAPI(lifespan=lifespan)

    @app.get('/download/{filename}')
    async def download_handler(filename: str):
        return prepare_and_serve_file(filename)

    # Montar aplicação Flet com configurações adequadas
    flet_app = flet_fastapi.app(
        main_app,
        upload_dir=upload_dir,
        secret_key=os.getenv("FLET_SECRET_KEY")
    )

    app.mount('/', flet_app)
    return app
```

Considerações de Segurança

1. **Validação de nomes de arquivo:** Sempre validar e sanitizar nomes de arquivos para evitar path traversal
2. **Autenticação:** Implementar controle de acesso aos endpoints de download se necessário
3. **Rate limiting:** Considerar limitação de taxa para endpoints de download
4. **Logs:** Implementar logging adequado para rastreamento de downloads

Esta abordagem fornece uma solução robusta, testada e que funciona consistentemente em diferentes ambientes de deployment, evitando os problemas comuns de reinicialização de aplicação, erros de rota e bloqueios do navegador.